

```
oNewAccount.deposit(100, userPassword) ❹
usersBalance = oNewAccount.getBalance(userPassword)
print()
print('After depositing 100, the user's balance is:', usersBalance)

# Show the new account
oNewAccount.show()
```

Listing 4-3: An extension of the test program to create an account on the fly

This test code asks the user for a name, a starting balance, and a password ❶. It uses these values to create a new account, and we store the newly created object in the variable `oNewAccount` ❷. We then call the `show()` method on the new object ❸. We deposit \$100 into the account and retrieve the new balance by calling the `getBalance()` method ❹. When we run the full program, we get the output from [Listing 4-2](#), as well as the following output:

```
What is the name for the new user account? Irv
What is the starting balance for this account? 777
What is the password you want to use for this account? IrvsPassword
    Name: Irv
    Balance: 777
    Password: IrvsPassword

After depositing 100, the user's balance is: 877
    Name: Irv
    Balance: 877
    Password: IrvsPassword
```

The key thing to notice here is that each `Account` object maintains its own set of instance variables. Each object (`oJoesAccount`, `oMarysAccount`, and `oNewAccount`) is a global variable that contains a collection of three instance variables. If we were to expand our definition of the `Account` class to include information such as address, telephone number, and date of birth, each object would get a set of these additional instance variables.